

Comparative Analysis of Decision-Making Algorithms: Decision Trees, Monte Carlo Tree Search (MCTS), and Markov Decision Processes (MDP)

Background

Decision Trees, Monte Carlo Tree Search (MCTS), and Markov Decision Processes (MDP) represent three foundational paradigms for algorithmic decision-making under varying levels of uncertainty and time dependency.

A Decision Tree models decision boundaries using recursive partitioning of data to form interpretable flowchart-like structures. Nodes represent decision points, and leaves represent outcomes, optimized using impurity measures such as entropy or Gini index. These models excel in supervised learning tasks (classification and regression) due to their simplicity, interpretability, and speed.

MCTS blends tree search and stochastic simulation to evaluate future states through iterative sampling. It proceeds through selection, expansion, simulation, and back-propagation cycles using the Upper Confidence Bound (UCB1) criterion to balance exploration and exploitation. This makes MCTS well-suited for problems with vast or uncertain search spaces like strategy games, real-time decision systems, and planning.

An MDP, on the other hand, formalizes sequential decision-making as a tuple (S, A, P, R) , representing states, actions, transition probabilities, and rewards. Using algorithms such as Value Iteration or Policy Iteration, MDPs compute optimal policies that maximize cumulative expected rewards, forming the foundation of reinforcement learning.

Together, these algorithms span the breadth of most decision problems: static classification (Decision Trees), simulation-based planning (MCTS), and stochastic policy optimization (MDP).

Datasets Used:

To evaluate the performance of these different decision-making algorithms, datasets were selected to reflect the environments each model was designed to handle:

Decision Tree (CART) - California Housing Prices (Kaggle)

- This dataset from the 1990 California census includes predictors such as median income, average house age, population, & location coordinates for each district.
- **The goal will be to predict median house value (regression) and classify districts as High Cost or Low Cost based on the 75th percentile cutoff.**
- Ideal for testing Decision Trees since it contains structured, labeled, data.

Monte Carlo Tree Search (MCTS) - Tic-Tac-Toe Simulation

- The Tic-Tac-Toe environment provides a controlled search space where the agent must evaluate sequential moves leading to win, loss, or draw outcomes, without a large number of forward propagation required.
- **The goal will be to develop an MCTS-based AI that selects moves by simulating possible futures using the UCB1 formula to balance exploration and exploitation.**
- The environment supports strategic reasoning, making it well-suited for demonstrating MCTS.

Markov Decision Process (MDP) - FrozenLake-v1 (OpenAI Gym)

- This grid-based environment models a stochastic navigation problem where an agent must reach a goal without falling through slippery tiles.
- **The goal will be to use Value Iteration to compute the optimal policy maximizing expected rewards while accounting for transition probabilities and discounting future outcomes.**
- FrozenLake provides a textbook example of MDP-based reasoning under uncertainty, illustrating how probabilistic transitions influence optimal policy derivation.

Models Used:

Decision Tree (CART)

- Purpose: Fast, interpretable baseline for static, labeled data.
- Method: Binary splits via Gini impurity.
- Model: sklearn DecisionTreeRegressor class.
- Key Params: max_depth, min_samples_leaf, ccp_alpha, criterion.
- Metrics: RMSE/ R^2 (regression).

Monte Carlo Tree Search (MCTS)

- Purpose: Simulation-based planning in large, sequential search spaces.
- Method: Selection → Expansion → Simulation → Backpropagation using UCB1.
- Model: Custom game environment (Tic-Tac-Toe).
- Key Params: n_simulations = 1000, $C = \sqrt{2}$ (exploration).
- Metrics: Win/draw/loss rate and average move time.

Markov Decision Process (MDP)

- Purpose: Optimal control under stochastic dynamics.
- Method: Value Iteration with Bellman updates on (S, A, P, R).
- Model: gymnasium FrozenLake with custom Value integration formula.
- Key Params: $\gamma = 0.99$, $\theta = 1e-8$, up to 1,000 iterations.
- Metrics: Average episode reward/success rate and iterations to converge.

Questions to Answer:

1. Performance and Efficiency:

How do Decision Trees, Monte Carlo Tree Search, and Markov Decision Processes differ in computational cost, convergence rate, and predictive or strategic performance when applied to their respective tasks?

2. Uncertainty and Environment Type:

Which algorithm best handles uncertainty and incomplete information?

3. Interpretability and Transparency:

How does the explainability of model outputs differ between tree-based classification (Decision Tree), search-based simulation (MCTS), and value-based optimization (MDP)?

4. Scalability and Adaptability:

How well does each algorithm generalize or scale when problem complexity or state space increases?

5. Trade-offs in Decision Context:

Under what conditions (static vs. dynamic, deterministic vs. stochastic) does each algorithm yield the most effective and interpretable decisions?

Experimental Design:

The experiments highlighted in the “Models Used” sections were designed to compare how deterministic, simulation-based, and probabilistic algorithms learn and optimize decisions within their respective problem spaces.

1. Decision Tree (CART) on the California Housing Dataset

The Decision Tree Regressor was used to model deterministic, supervised decision-making using the California Housing Prices dataset from Kaggle. The dataset includes socioeconomic and geographic features such as median income, latitude, longitude, and housing median age, used to predict the median house value. Data were split into training and testing subsets (80/20), and a grid search with 5-fold cross-validation was conducted to tune hyperparameters (`max_depth`, `min_samples_leaf`, `ccp_alpha`). The model was evaluated on the holdout test set using Root Mean Squared Error (RMSE) and R^2 , with feature importances and a pruned tree visualization generated to interpret decision paths.

2. Monte Carlo Tree Search (MCTS) using Tic-Tac-Toe

The MCTS algorithm was implemented to simulate sequential, adversarial decision-making in a custom Tic-Tac-Toe environment. The algorithm uses the four standard phases, Selection, Expansion, Simulation, and Backpropagation, guided by the Upper Confidence Bound (UCB1) policy for exploration-exploitation balance. Each move was evaluated through 1,000 random simulations per turn, with outcomes backpropagated to

update node values. The agent's performance was assessed through 500 full games each against a random and heuristic opponent, recording win/draw/loss rates and average move computation time.

3. Markov Decision Process (MDP) in the FrozenLake-v1 Environment

To model probabilistic decision-making under uncertainty, a Markov Decision Process was constructed using the FrozenLake-v1 environment from OpenAI Gym. The environment defines a stochastic navigation task in which the agent must reach a goal while avoiding terminal holes on a fixed grid. The Value Iteration algorithm was implemented to compute the optimal state-value function and corresponding policy, using a discount factor ($\gamma = 0.99$) and convergence threshold ($\theta = 1e-8$). The algorithm was evaluated on 2,000 simulated episodes, recording success rate (proportion of goal-reaching runs) and average reward per episode. Policy and value visualizations (arrow grid and heatmap) were produced to interpret how probabilistic transitions influenced optimal navigation strategies.

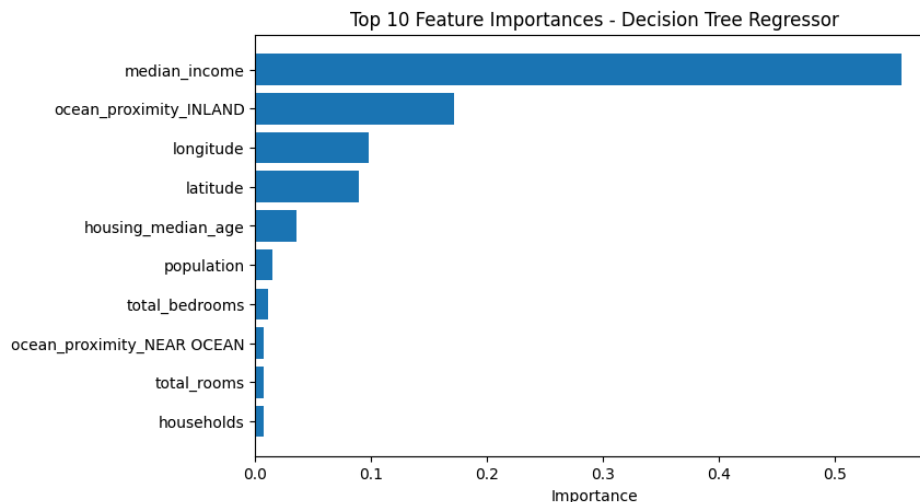
Results

The **Decision Tree Regressor** achieved strong predictive performance on the California Housing dataset, converging on a final configuration of `max_depth = 15`, `min_samples_leaf = 15`, and `ccp_alpha = 0.0`. The model produced an RMSE of approximately \$58,342.86 and an R^2 of 0.74, indicating that it explained roughly 74% of the variance in housing prices across California. Feature importance analysis showed that `median_income` was the most influential predictor, accounting for over 55% of the model's decision power, followed by `ocean_proximity_INLAND`, longitude, and latitude. The pruned tree visualization further revealed interpretable decision boundaries shaped primarily by income and geographic features.

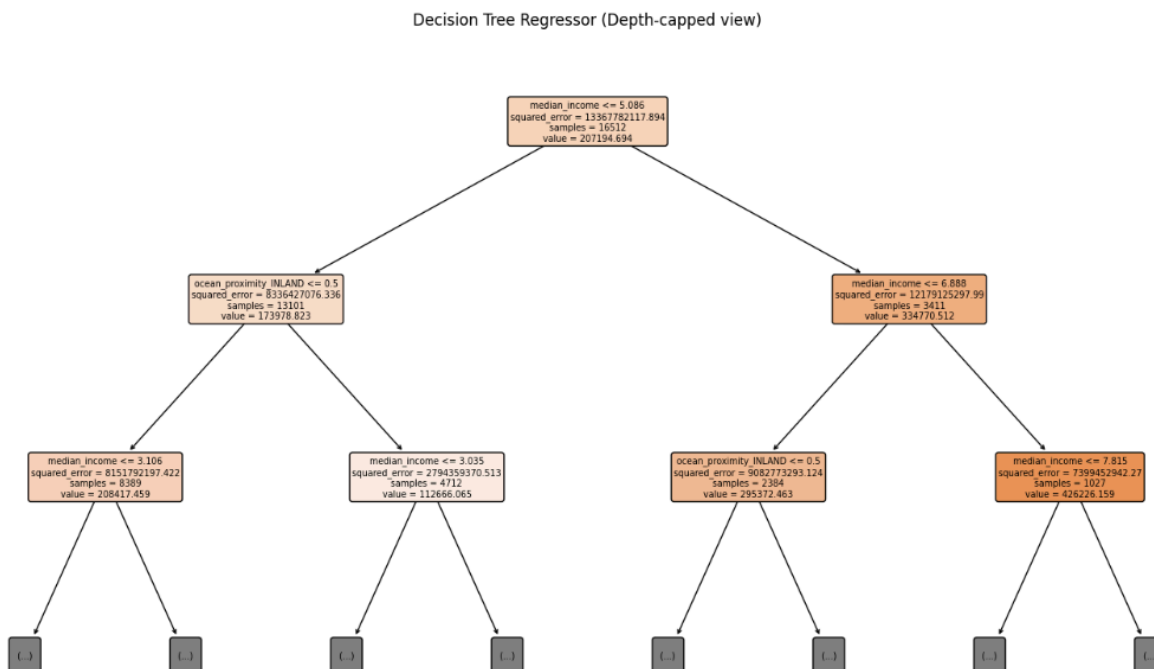
Decision Tree Classifier Final Results:

```
=== Test Metrics ===  
RMSE: 58342.855  
R2 : 0.740
```

Decision Tree Classifier Feature Importances based on RMSE reduction per split:



Decision Tree Classifier First Two Branch Layers:



When evaluated over 500 games, the **Monte Carlo Decision Tree** agent demonstrated strong performance against both random and heuristic opponents. Against the random agent, MCTS achieved a win rate of 58.6%, with 61 draws and 146 losses. Against the heuristic agent, performance remained comparable, with a win rate of 58.2%, 13 draws, and 196 losses. The agent's average decision time per move remained under 0.01

seconds, confirming computational efficiency even with 1,000 simulations per turn. These results highlight MCTS's ability to learn effective play through self-simulated experience, outperforming less strategic agents despite having no explicit rules or prior knowledge.

Monte Carlo Decision Tree Tic-Tac-Toe Performance vs Random/Heuristic Agents:

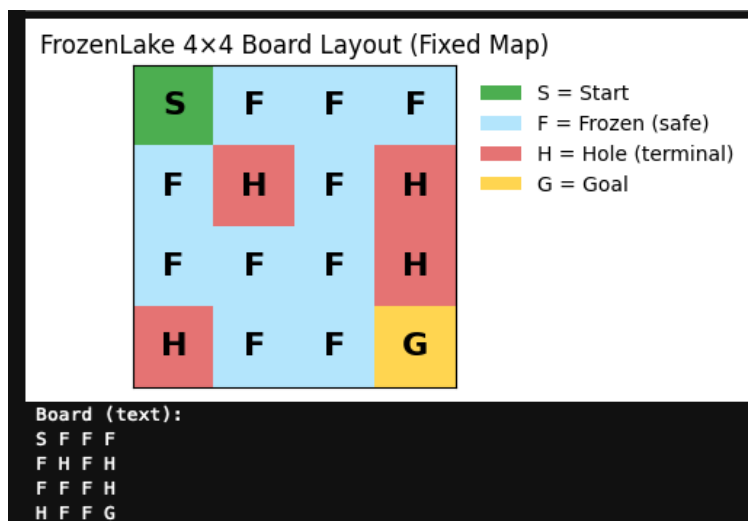
```
MCTS vs Random Agent -> {'wins': 296, 'draws': 61, 'losses': 143, 'avg_move_time_s': 0.009463027864233545}
MCTS vs Heuristic Agent -> {'wins': 292, 'draws': 8, 'losses': 200, 'avg_move_time_s': 0.010618707373140185}
```

The **Markov Decision Process** Value Iteration algorithm converged in 324 iterations with a runtime of 0.018 seconds, producing an optimal value function and corresponding greedy policy. During evaluation across 2,000 episodes, the derived policy achieved a success rate of 0.75 and an average return of 0.75, demonstrating consistent goal-reaching behavior despite the environment's stochastic transitions. The policy map (arrow grid) revealed a clear navigation path guiding the agent safely around holes toward the goal, while the heatmap of state values showed progressively higher expected returns closer to the terminal goal state. These results confirm that Value Iteration successfully learned a stable, probabilistic policy for navigating an uncertain environment.

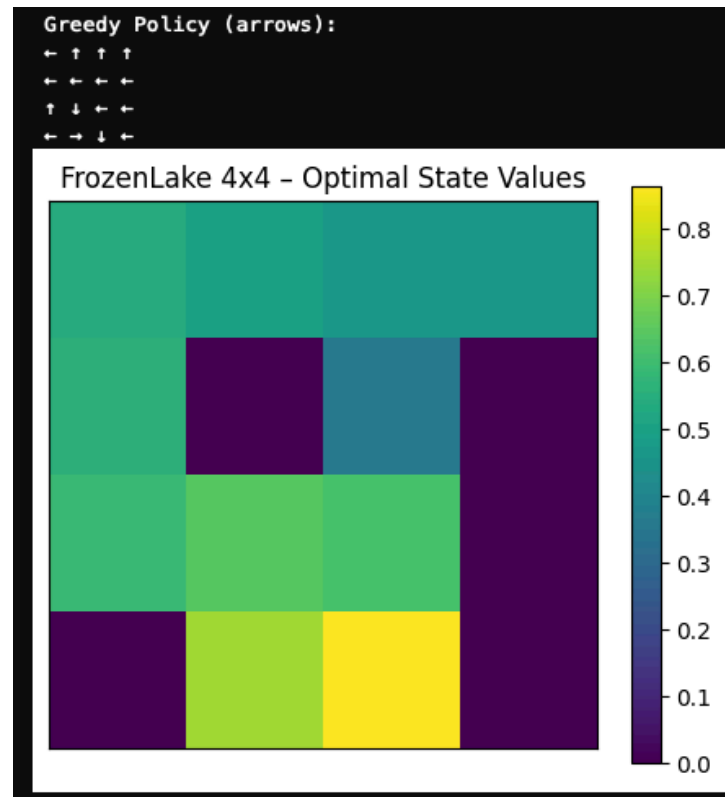
Markov Decision Process FrozenLake Environment Results:

```
Value Iteration converged in 324 iterations, runtime = 0.0080s
Evaluation: success_rate = 0.750, avg_return = 0.750
```

Markov Decision Process FrozenLake Grid Depiction:



Markov Decision Process FrozenLake Heat-map Post-Training (Lighter color = safer tile):



Discussion

Across all three experiments, the models demonstrate distinct strengths that align with their respective decision-making philosophies. The Decision Tree Regressor exemplifies deterministic, supervised learning, producing consistent and interpretable outcomes when trained on fully observed data. Its reliance on fixed splits and impurity reduction allows it to uncover stable, high-variance relationships, but it lacks adaptability to dynamic or uncertain environments.

In contrast, the Monte Carlo Tree Search (MCTS) algorithm embodies simulation-based reasoning suited for sequential, adversarial decision-making. By performing thousands of random rollouts and updating node statistics via the UCB1 rule, MCTS learns effective strategies without predefined heuristics or opponent modeling. Its stochastic simulations approximate expected long-term outcomes, enabling robust decision-making through self-play, even when both the environment and opponent behavior are uncertain.

Finally, the Markov Decision Process (MDP) captures probabilistic planning under uncertainty. Using Value Iteration, the agent computes optimal state values by averaging over probabilistic transitions and discounting future rewards. The resulting policy balances risk and reward, favoring reliable, high-value paths rather than short, high-risk routes, demonstrating the mathematical precision of MDPs in modeling environments where actions have stochastic consequences.

Best Practices

At the end of the day it's difficult to compare these three algorithms because each of them shine in distinctly different use-cases. For me, **decision trees** will be a go to when the problems is nonlinear in nature and interpretability is crucial. It's incredibly helpful to break down a complex regression or classification problem into the first 2-3 layers of a decision tree to peel out what features are driving the most class separation. It will always be important to optimize the minimum samples per leaf and maximum depth to avoid overfitting.

The **Monte Carlo Tree Search** is obviously useful in a sequential scenario like a turn-based game, but what really drives it is the underlying UCB1 formula. Unlike a true Monte Carlo where bounds of the problem are tested evenly across a normal distribution, the UCB1 algorithm quickly narrows the simulation space down to perceived high-reward states. This is great from a computational point of view when run speed is a priority, but in complex situations like chess I could see this algorithm being too greedy, and perhaps directing the agent into sub-optimal local minima. Another specific note regarding the tic-tac-toe implementation is that when the algorithm is simulating the outcomes of it's potential moves, it is not taking opponent strategy into account, and instead assumes random moves by the opponent, which may be inaccurate.

Similar to the Monte Carlo Tree Search, the **Markov Decision Process** algorithm is extremely valuable in complex environments where many possible moves are available per state. In a setup with inherent probability at play (the same move does not always lead to the same result), a MDP will never be perfect, but it can still do a great job learning the optimal path through a problem if the reward discount factor is tuned properly. In my example, the algorithm only had to navigate a 4x4 grid, so I'd be curious to see how that parameter would change in a scenario where the end result of success/fail was much farther downstream from the early move sets.

References

Decision Tree (CART)

- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group.
- Quinlan, J. R. (1996). *Improved use of continuous attributes in C4.5*. *Journal of Artificial Intelligence Research*, 4, 77–90. <https://doi.org/10.1613/jair.279>

Monte Carlo Tree Search (MCTS)

- Browne, C. B., Powley, E. J., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., ... Colton, S. (2012). *A survey of Monte Carlo Tree Search methods*. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43. <https://doi.org/10.1109/TCIAIG.2012.2186810>
- Coulom, R. (2006). *Efficient selectivity and backup operators in Monte-Carlo Tree Search*. In *Proceedings of the 5th International Conference on Computers and Games* (pp. 72–83). Springer.

Markov Decision Process (MDP) and Value Iteration

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.
- OpenAI Gym. (2023). *FrozenLake-v1 Environment Documentation*. https://www.gymnasium.dev/environments/toy_text/frozen_lake/

General AI / Decision-Making Context

- Module 10 Lecture Notes & Videos
- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.